

V – IHK الگوی اصلی (اسکلت شبه‌کد)

فقط فرم کلی الگوریتم‌ها برای تمرین سریع در این فایل است. برای توضیح و مثال‌های واقعی از فایل «Kompakt» استفاده کن.

Muster 1 – Suche / enthält (boolean)

جستجو در آرایه / لیست

```
funktion enthaelt(feld: array<T>, gesucht: T) : boolean
  n = laenge(feld)
  für i = 0 bis n - 1
    wenn feld[i] = gesucht dann
      return true
    ende wenn
  ende برای
  return false
end funktion
```

شمردن تعداد عناصر با شرط

```
funktion zaehleMitBedingung(feld: array<T>) : integer
    n = laenge(feld)
    anzahl = 0
    für i = 0 bis n - 1
        wenn <bedingung für feld[i]> dann
            anzahl = anzahl + 1
        ende wenn
    ende برای
    return anzahl
end funktion
```

```
funktion erstelleListeMitBedingung(feld: array<T>)
    : liste<T>

    liste = neueListe()
    n = laenge(feld)
    für i = 0 bis n - 1
        wenn <bedingung für feld[i]> dann
            liste.hinzufuegen(feld[i])
        ende wenn
    ende برای
    return liste
end funktion
```

حداقل، حداکثر، میانگین، طولانی‌ترین دنباله

```
funktion findeMin(werte: array<integer>) : integer
    n = laenge(werte)
    min = werte[0]
    für i = 1 bis n - 1
        wenn werte[i] < min dann
            min = werte[i]
        ende wenn
    ende برای
    return min
end funktion
```

```
funktion findeMax(werte: array<integer>) : integer
    n = laenge(werte)
    max = werte[0]
    für i = 1 bis n - 1
        wenn werte[i] > max dann
            max = werte[i]
        ende wenn
    ende برای
    return max
end funktion
```

```
funktion mittelwert(werte: array<integer>) : double
    n = laenge(werte)
    summe = 0
    für i = 0 bis n - 1
        summe = summe + werte[i]
    ende برای
    return summe / n
end funktion
```

```
funktion laengsteFolgeMitBedingung(werte: array<integer>)
                                     : integer

    aktuelleLaenge = 0
    besteLaenge = 0
    n = laenge(werte)

    für i = 0 bis n - 1
        wenn <bedingung für werte[i]> dann
            aktuelleLaenge = aktuelleLaenge + 1
            wenn aktuelleLaenge > besteLaenge dann
                besteLaenge = aktuelleLaenge
            ende wenn
        sonst
            aktuelleLaenge = 0
        ende wenn
    ende برای

    return besteLaenge
```

end funktion

Prüfziffer (وزندهی) و تبدیل عدد

```
funktion pruefziffer(ziffern: array<integer>,
                    gewichte: array<integer>) : integer
    summe = 0
    n = laenge(ziffern)
    g = laenge(gewichte)

    für i = 0 bis n - 1
        summe = summe + ziffern[i] * gewichte[i mod g]
    ende برای

    return summe mod 10      // یا mod 11
end funktion
```

```
funktion dezimalZuBinaer(n: integer) : string
    wenn n = 0 dann
        return "0"
    ende wenn

    ergebnis = ""
    wert = n

    solange wert > 0
        rest = wert mod 2
        ergebnis = konvertiere(rest) + ergebnis
        wert = wert / 2
    ende solange

    return ergebnis
end funktion
```

مرتب‌سازی ساده آرایه

```
prozedur sortiereNachFeld(feld: array<T>)  
  n = laenge(feld)  
  für i = 0 bis n - 2  
    für j = 0 bis n - 2 - i  
      wenn feld[j] > feld[j + 1] dann  
        tausche feld[j], feld[j + 1]  
      ende wenn  
    ende برای  
  ende برای  
end prozedur
```

```
prozedur sortiereObjekteNachKey(obj: array<T>)  
  n = laenge(obj)  
  für i = 0 bis n - 2  
    für j = 0 bis n - 2 - i  
      wenn obj[j].key > obj[j + 1].key dann  
        tausche obj[j], obj[j + 1]  
      ende wenn  
    ende برای  
  ende برای  
end prozedur
```

مرتب‌سازی با comparator

```
prozedur sort(liste: array<T>,
              cmp: funktion(T, T): integer)
  n = laenge(liste)
  für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
      wenn cmp(liste[j], liste[j + 1]) > 0 dann
        tausche liste[j], liste[j + 1]
      ende wenn
    ende برای
  ende برای
end prozedur
```

```
funktion vergleicheObjekt(o1: T, o2: T) : integer
  wenn <o1.key kleiner> dann
    return -1
  sonst wenn <o1.key groesser> dann
    return 1
  sonst
    return 0
  ende wenn
end funktion
```

آرایهٔ دوبعدی و آمار

```

funktion statistik2D(feld: array<array<integer>>,
                    limit: integer) : statistik
    min = +unendlich
    max = -unendlich
    liste = neueListe()

    zeilen = laenge(feld)

    für i = 0 bis zeilen - 1
        hatLimit = false
        spalten = laenge(feld[i])

        für j = 0 bis spalten - 1
            wert = feld[i][j]

            wenn wert < min dann
                min = wert
            ende wenn
            wenn wert > max dann
                max = wert
            ende wenn
            wenn wert > limit dann
                hatLimit = true
            ende wenn
        ende برای

        wenn hatLimit dann
            liste.hinzufuegen(i)
        ende wenn
    ende برای

    return neueStatistik(min, max, liste)
end funktion

```

```

funktion baueMatrix(tage: integer, stunden: integer)
    : array<array<integer>>
    matrix = neuesArray2D(tage, stunden, 0)
    // hier: matrix[i][j] nach Bedarf fuellen
    return matrix
end funktion

```